Memorandum

To: Charlie Refvem, Department of Mechanical Engineering, Cal Poly SLO
crefvem@calpoly.edu

From: Lucas Nathan
lrnathan@calpoly.edu

Vandan Patel
vpatel43@calpoly.edu

Oliver Lenoir
olenoir@calpoly

Date: 24 February 2026

RE: **Lab 0x05 – Line Following**

Overview

This memo summarizes the major design decisions implemented during Lab 0x05, focusing on three primary areas: the line sensor hardware driver design, modifications to the cooperative task structure and shared variables, and enhancements to the user interface to support automated data collection. The objective of these changes was to create a modular, scalable system capable of automated step response testing while preserving reliable closed-loop motor control and preparing the architecture for future line-following functionality.

Our implementation emphasized clean separation between hardware drivers, control logic, and user interaction. Hardware-specific operations were isolated within dedicated driver classes, motor control tasks were kept largely unchanged to maintain stability, and shared variables were structured to allow safe communication between tasks. The user interface was redesigned to enable structured command-based interaction, floating-point gain entry, and automated step response execution with formatted data output suitable for PC-based logging and plotting.

Collectively, these design choices produced a flexible and extensible system architecture that supports efficient controller tuning, repeatable automated testing, and future expansion in subsequent labs.

Overview of Program Flow

In Lab 0x05, our team expanded our closed-loop motor control system to support automated step response testing, structured user interaction, and improved modularity in preparation for future line-following implementation. The primary objective was to enable dynamic gain tuning and setpoint selection through a serial interface while maintaining stable cooperative multitasking and reliable motor control performance.

To achieve this, we preserved the core motor control tasks from Lab 0x03 and layered additional functionality through a dedicated user interface task and structured shared-variable communication. The motor tasks remain responsible for encoder reading, velocity calculation, PI control, and motor actuation. A separate UI task manages user commands, gain entry, setpoint selection, and step-response triggering. Communication between tasks is handled using Share objects for controller parameters and control flags, and Queue objects for storing time-series step response data. This structure maintains clean separation between control logic and user interaction while preventing unintended task interference.

We also designed the system with hardware abstraction and scalability in mind. The line sensor was implemented as a standalone driver class responsible for ADC initialization, multi-channel readings, and calibration handling. By encapsulating sensor functionality within a dedicated module, we reduced coupling between hardware access and higher-level control logic, improving maintainability and future expandability.

The user interface was redesigned into a structured, command-based system that supports multi-character floating-point input for gains and setpoints. When a step response is triggered, the system automatically collects and prints formatted CSV-style data, ensuring compatibility with a PC-based Python script for automated logging and plotting. This automation eliminates manual data transcription and significantly improves testing efficiency.

Overall, our design emphasizes modularity, cooperative multitasking integrity, structured task communication, and automated testing capability. These improvements establish a scalable software architecture suitable for continued development in subsequent labs.

Appendix A. Diagrams

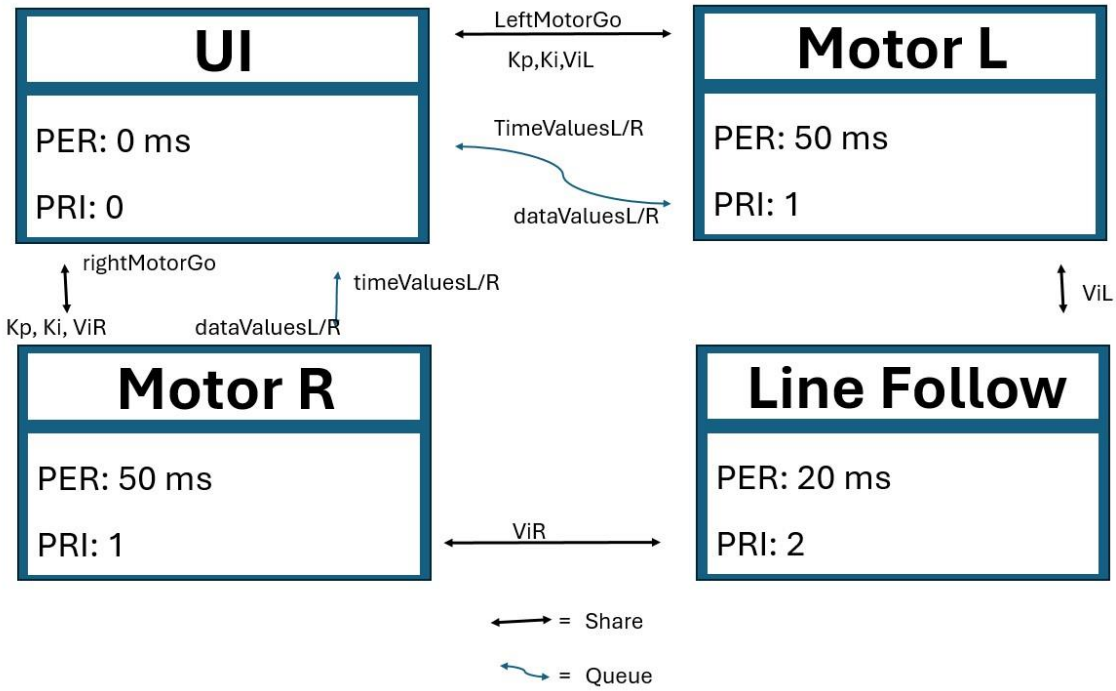

Figure A-1. Task Diagram. Shows how the four tasks communicate with each other and the scheduler.

The UI period and priority were chosen with the intent of the UI running at lowest priority, whenever the task manager had downtime.

Both motors having the same priority is to ensure symmetric control behavior

Figure A-2. Motor Task Finite State Machines (same for both left and right).

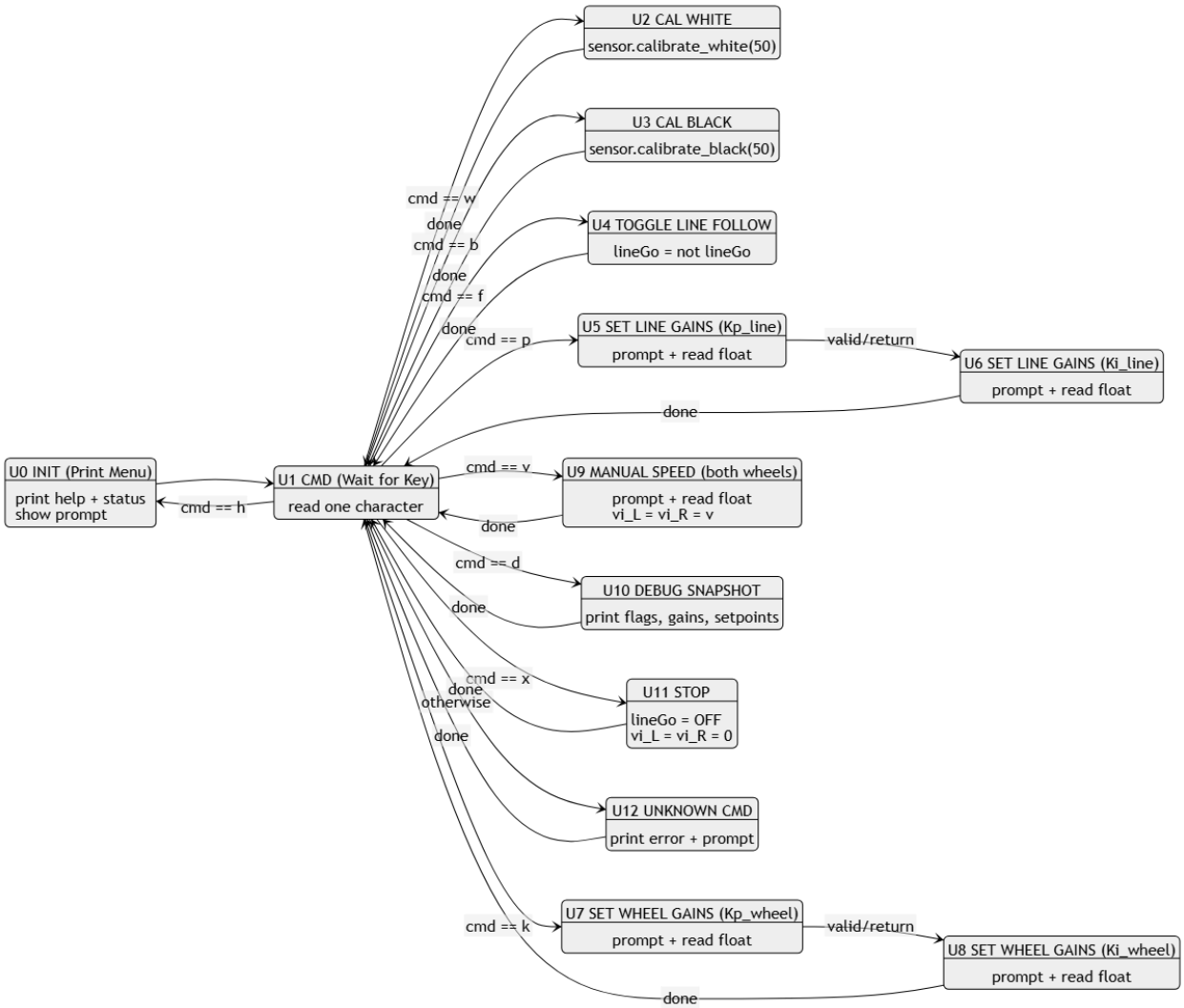

Figure A-3. User Interface Finite State Machine.

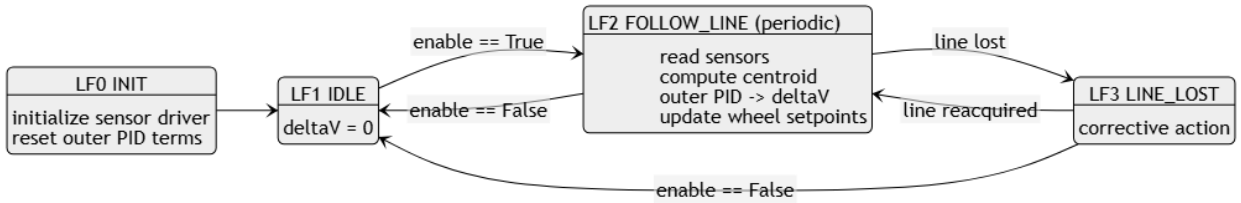

Figure A-4. Line Follow Finite State Machine

Appendix B. Plots

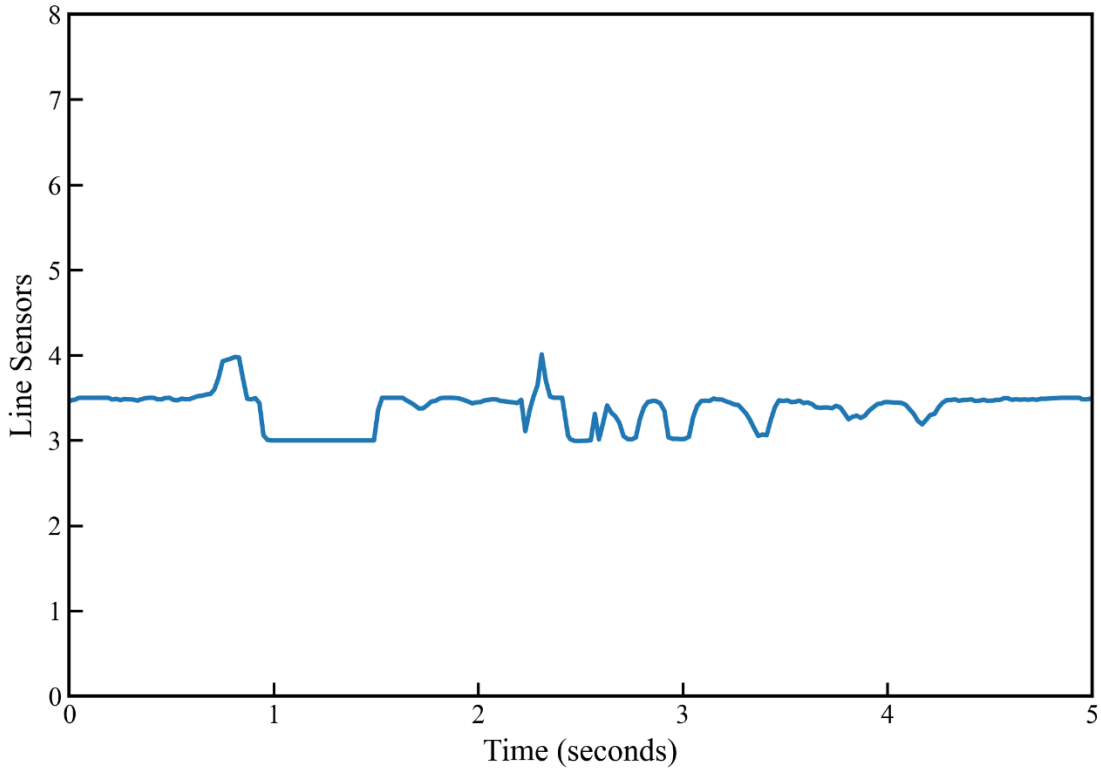

Figure B-1. Plot of line following performance.

Figure B-1 shows the measured line position over a 5-second interval during closed-loop line following. The plot demonstrates that the controller maintains the detected line near the center position for the duration of the test. While small fluctuations are present, the signal remains bounded and does not exhibit drift or instability.

Overall, the graph indicates that the line-following controller successfully tracks the line and maintains stable operation over time.

```python

            yield

    def _print_menu(self):
        self._ser.write("\r\n+-------------------------------------------------------------+\r\n")
        self._ser.write("| ME 405 Romi Line Following UI (Lab 0x05)                    |\r\n")
        self._ser.write("+---+---------------------------------------------------------+\r\n")
        self._ser.write("| h | Help menu                                                |\r\n")
        self._ser.write("| w | Calibrate WHITE (on white surface)                       |\r\n")
        self._ser.write("| b | Calibrate BLACK (on black line)                          |\r\n")
        self._ser.write("| f | Toggle line following ON/OFF                             |\r\n")
        self._ser.write("| p | Set line gains (Kp_line, Ki_line)                        |\r\n")
        self._ser.write("| k | Set wheel gains (Kp_wheel, Ki_wheel)                     |\r\n")
        self._ser.write("| v | Set forward speed (base speed for line follow)           |\r\n")
        self._ser.write("| t | Dump centroid vs time (CSV)                              |\r\n")  # <-- NEW
        self._ser.write("| d | Debug: print latest error/integral/dv                    |\r\n")
        self._ser.write("| x | STOP (0 speed + line follow OFF)                         |\r\n")
        self._ser.write("+---+---------------------------------------------------------+\r\n\r\n")

        self._ser.write("Status: line_follow=" + ("ON" if self._lineGo.get() else "OFF") + "\r\n")
        self._ser.write("Line gains:  Kp=" + str(self._kpline.get()) + "  Ki=" + str(self._kiline.get()) + "\r\n")
        self._ser.write("Wheel gains: Kp=" + str(self._kpw.get())   + "  Ki=" + str(self._kiw.get()) + "\r\n")
        self._ser.write("BaseV(line): " + str(self._baseLineV.get()) + " m/s\r\n")  # <-- NEW
        self._ser.write("vi_L=" + str(self._viL.get()) + "  vi_R=" + str(self._viR.get()) + "\r\n\r\n")
        self._ser.write(UI_prompt)

    def _stop_robot(self):
        self._lineGo.put(False)
        self._viL.put(0.0)
        self._viR.put(0.0)

    def run(self):
        while True:
            if self._state == S0_INIT:
                self._print_menu()
                self._state = S1_CMD

```

Figure B-2. Updated UI for Putty

Appendix C. Functionality Demonstration

A video is attached to this submission showing the ROMI following the line for multiple revolutions of the circle.

Appendix D. Source Code

main.py

This is the main entry point of the project. It initializes hardware components, creates system tasks, and starts the cooperative task scheduler. It serves as the central controller that brings the entire embedded system together and begins program execution.

cotask.py

This file implements the cooperative multitasking scheduler. It defines task behavior, manages task priorities and states, and runs the scheduling loop. It allows multiple tasks to operate concurrently without preemption.

task_share.py

This file manages shared variables and queues used for communication between tasks. It ensures data can be safely passed between different parts of the system without corruption during multitasking.

task_motor.py

This file contains the motor control task. It likely implements the control algorithm (such as PID control), reads encoder feedback, and determines the appropriate motor command. It manages the overall closed-loop motor behavior.

motor_driver.py

This file provides the low-level interface to the motor hardware. It sets PWM signals, controls motor direction pins, and abstracts the hardware details so higher-level code can control the motor more easily.

encoder.py

This file handles reading the quadrature encoder signals. It tracks motor position and possibly speed, providing feedback data that is used for closed-loop motor control.

task_user.py

This file manages user interaction. It likely handles serial communication, processes user commands, and displays system information or test results. It acts as the interface between the user and the embedded system.

PC code.py

This file runs on a computer and communicates with the embedded system, typically through serial communication. It may send control commands, receive system data, log performance results, and generate plots for analysis.

line_sensor.py

This file implements the hardware driver for the line sensor array. It provides methods for reading raw sensor values, performing calibration (white and black references), and computing processed outputs such as normalized readings or centroid position. This abstraction allows higher-level tasks to access line position data without directly managing hardware details.

task_line.py

This file contains the line-following task. It reads processed data from the line sensor driver, computes the centroid error relative to the center of the sensor array, and applies an outer PID controller to generate a steering correction. The resulting correction is applied as a differential adjustment to the left and right wheel velocity setpoints, forming the outer loop of the cascaded control structure.